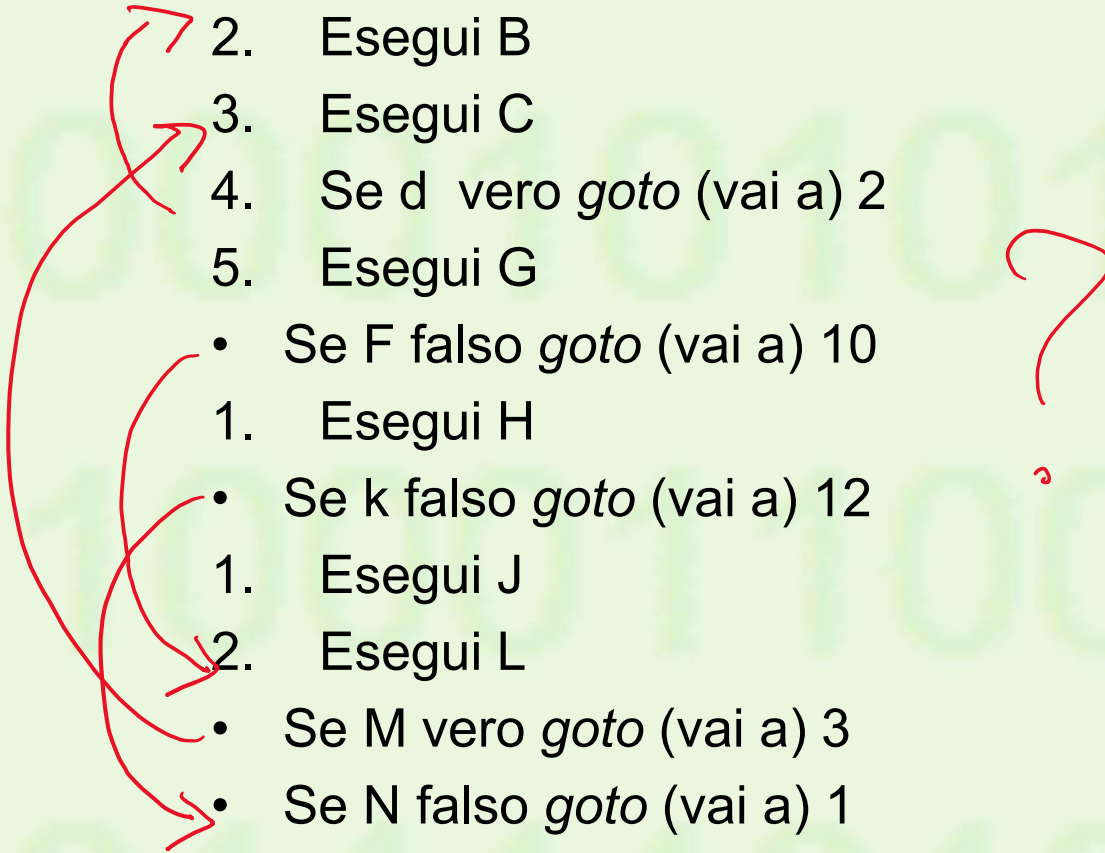


Laboratorio di Programmazione

Lezione 4-5

Esempio di programma con goto

1. Esegui A
2. Esegui B
3. Esegui C
4. Se d vero *goto* (vai a) 2
5. Esegui G
 - Se F falso *goto* (vai a) 10
1. Esegui H
 - Se k falso *goto* (vai a) 12
1. Esegui J
2. Esegui L
 - Se M vero *goto* (vai a) 3
 - Se N falso *goto* (vai a) 1
1.
2.



Programmazione strutturata

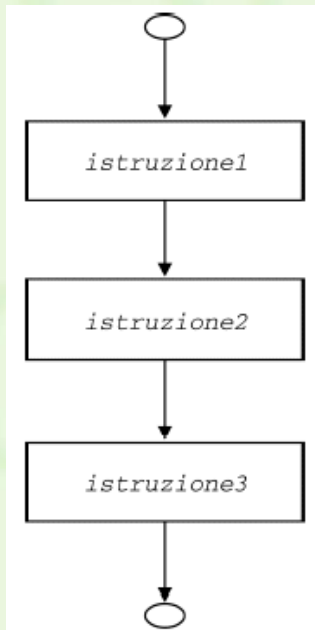
Paradigma di programmazione che fa uso di:

- **Strutture di controllo:**
 - Sequenza (succesione finita di istruzioni);
 - Selezione (scelta tra possibili istruzioni)
 - Iterazione/ricorsione (ripetizione blocco di istruzioni)
- **Componibilità delle strutture di controllo**

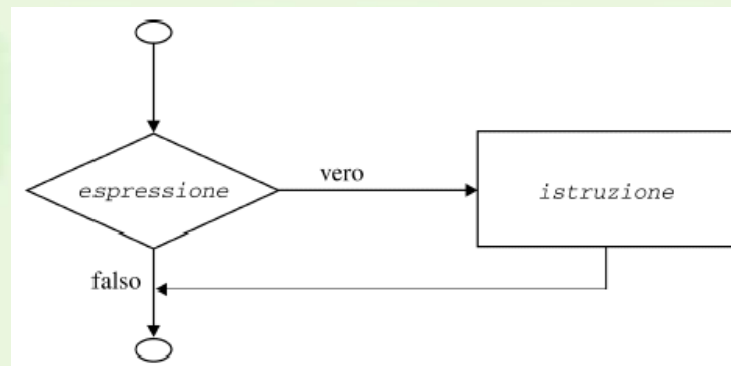
L'obiettivo della *programmazione strutturata* è di rendere un flusso ordinato il passaggio tra le istruzioni dall'inizio alla fine dei programmi.

Teorema di Böhm- Jacopini (1966)

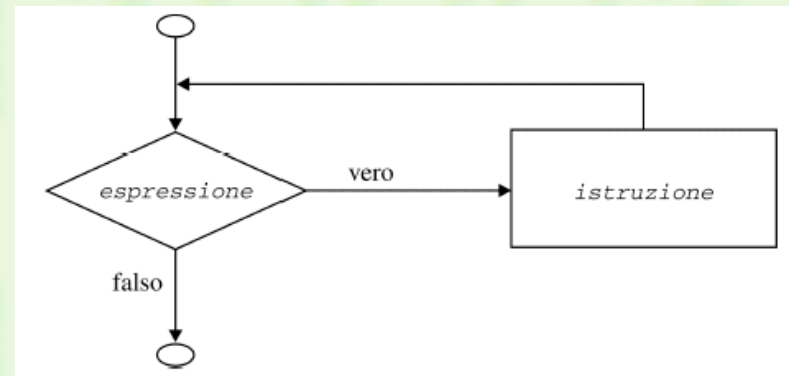
Corrado Böhm e Giuseppe Jacopini hanno dimostrato che tutti i programmi possano essere scritti con l'utilizzo di: *sequenza*, *selezione* e *iterazione* (senza l'uso del salto *goto*).



Sequenza



Selezione



Iterazione

Comandi condizionali o di selezione

- Esprimono **un'alternativa** fra due o più possibili prosecuzioni della computazione sulla base della *valutazione di una espressione logica*

Comandi iterativi

- Permettono di esprimere computazioni che si *ripetono* un numero di volte che è variabile.
- **Iterazione determinata**: con costrutti che permettono di iterare un *numero fissato di volte*;
- **Iterazione indeterminata**: con costrutti che permettono di iterare fino al *verificarsi di una determinata condizione*;

(La ricorsione)

- Metodo alternativo all'iterazione in cui un processo di calcolo richiama se stesso (la vedremo tra un po' di lezioni...)

L'istruzione **if**

- Sintassi

```
if ( <espressione booleana> ){  
    <istruzione_1>;  
    .....  
    <istruzione_r>;  
}  
  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

Il blocco <istruzione_1>..*<istruzione_r>* viene eseguito **soltanto** se <espressione booleana> è *true*

Istruzione **while**

- Sintassi

```
while (<espressione booleana>){  
    <istruzione_1>  
    ...  
    <istruzione_r>  
}  
  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

Il blocco <istruzione_1>..**<istruzione_r>** viene eseguito **fintanto che** <espressione booleana> è **true**

1) Iterazione definita : ciclo con "contatore"

numero di iterazioni conosciuto a-priori

- Utilizzo una variabile per contare:
 - La inizializzo a 0 prima del ciclo
 - All'interno del ciclo la incremento
 - La condizione del while controlla quando arrivo al valore scelto

2) Un ciclo a iterazione "indefinita"

numero di iterazioni NON conosciuto a-priori

- La condizione del while determina il numero di volte che il ciclo viene eseguito .. (può dipendere dalla particolare esecuzione)

Esempio di iterazione definita:

Calcolare la somma di 50 numeri immessi dall'utente.

```
int a, s, c;  
s=0;  
  
c=0;  
while( c < 50 ){  
    printf("scrivi il numero\n");  
    scanf("%d", &a);  
    s=s+a;  
    c=c+1;  
}  
  
printf("la somma dei numeri è %d", s);
```

Esempio di iterazione indefinita:

Calcolare la somma di alcuni numeri immessi dall'utente.
L'immissione termina quando l'utente inserisce 0

```
int a, s;  
s=0;  
  
a=1;  
while( a !=0 ){  
    printf("scrivi il numero\n");  
    scanf("%d", &a);  
    s=s+a;  
  
}  
  
printf("la somma dei numeri è %d", s);
```

Esempio di istruzione if dentro un while:

Calcolare quanti numeri pari sono presenti tra 50 numeri immessi dall'utente.

```
int a, p, c;  
p=0;  
c=0;  
  
while( c < 50 ){  
    printf("scrivi il numero\n");  
    scanf("%d", &a);  
    if(a%2==0)  
        p=p+1;  
    c=c+1;  
}  
  
printf("Sono stati immessi %d numeri pari", p);
```

Istruzione **while**

(repetita iuvant)

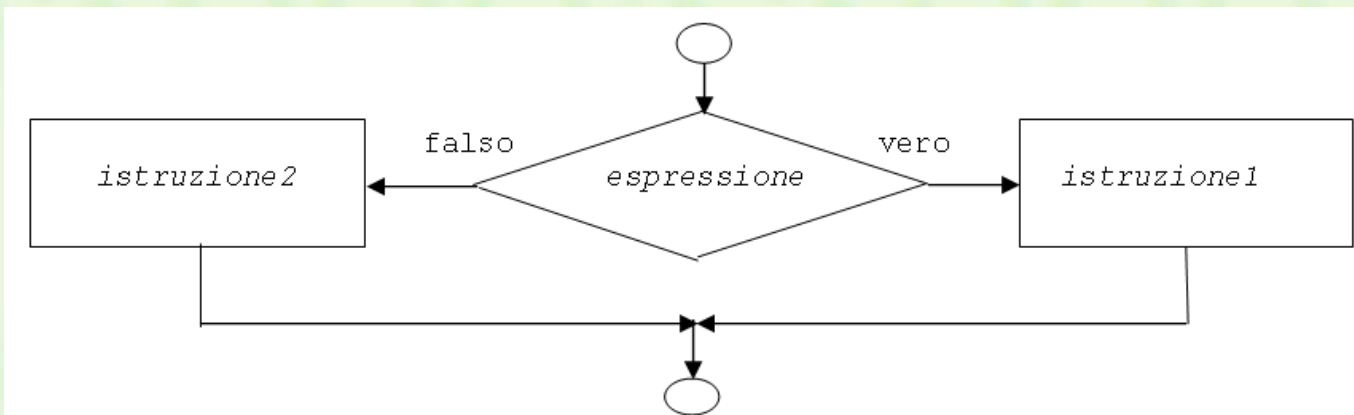
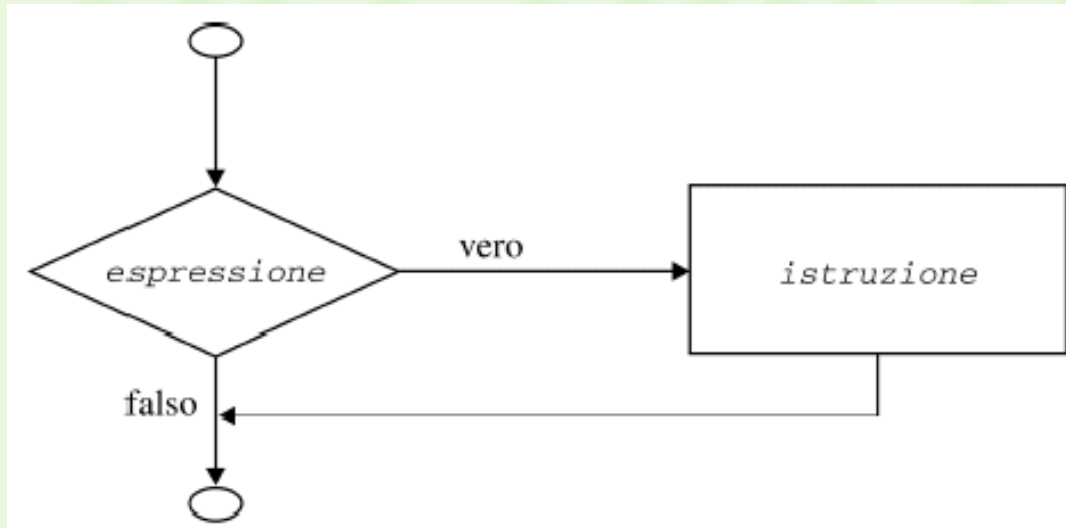
ATTENZIONE!!!

Tra le istruzioni del corpo del **while** ci deve essere qualcosa che coinvolge e modifica le variabili della **<espressione booleana>** e la rende **false**

Altrimenti... LOOP INFINITO

Esistono altre istruzioni di selezione e di iterazione....

Due possibili schemi per la selezione:



L'istruzione **if**

- Sintassi

```
if ( <espressione booleana> ){  
    <istruzione_1>;  
    .....  
    <istruzione_r>;  
}  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

Il blocco <istruzione_1>..*<istruzione_r>* viene eseguito **soltanto** se <espressione booleana> è *true*

L'istruzione **if...else**

- Sintassi

```
if ( <espressione booleana> )  
    { <istruzione_1>;  
      .....  
      <istruzione_r>; }  
else  
    { <istruzione_r+1>;  
      .....  
      <istruzione_s>; }  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

SE <espressione booleana> è **true** viene eseguito Il blocco
<istruzione_1>..**ALTRIMENTI** <istruzione_r> viene eseguito Il blocco
<istruzione_r+1>..**ALTRIMENTI** <istruzione_s>;

Problema del *dangling else*

```
int main (void)
{
    int x=3,y=4;
    if (x==0)
        if (y<0)
            x=y;
    else
        x=x-y;
}
```

- Quanto vale x?

A quale if è collegato
l'else?

Ogni else è collegato all'if
più vicino
che ne è sprovvisto

PROBLEMA 1:

Scrivere un programma che prende in **input 2 numeri interi** inseriti da tastiera e li scrive in **output in ordine crescente** utilizzando in totale soltanto **1 confronto** (con operatore $\leq$) tra i numeri stessi .

No variabili supplementari (solo **2 variabili per i 2 numeri**).

Soluzione: max2.c

```
//max2.c
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int x, y;
```

```
printf ("Scrivi il primo numero  x: ");
```

```
scanf ("%d", &x);
```

```
printf ("Scrivi il secondo  numero y:  ");
```

```
scanf ("%d", &y);
```

```
if (x > y)
```

```
    printf ("I numeri in ordine sono y=%d - x=%d  \n", y,x);
```

```
else
```

```
    printf ("I numeri in ordine sono x=%d - y=%d  \n", x,y);
```

```
return(0);
```

```
}
```

PROBLEMA 2:

Scrivere un programma che prende in **input 3 numeri interi** inseriti da tastiera e calcola il maggiore utilizzando in totale soltanto **2 confronti** (con operatore $\leq$) tra i numeri stessi all'interno di istruzioni if-else annidate.
No variabili supplementari (solo **3 variabili per i 3 numeri**).

Soluzione: max3.c

```
//max3.c
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int num1, num2, num3;
```

```
printf ("Scrivi il primo numero  ");
```

```
scanf ("%d", &num1);
```

```
printf ("Scrivi il secondo  numero  ");
```

```
scanf ("%d", &num2);
```

```
printf ("Scrivi il terzo numero  ");
```

```
scanf ("%d", &num3);
```

<continua>

....

```
if (num1 > num2)
    { if (num1 > num3)
        printf ("Il piu' grande e' il primo! \n\n ");
      else
        printf ("Il piu' grande e' il terzo! \n\n ");
    }
else
    {if (num2 > num3)
        printf ("Il piu' grande e' il secondo! \n\n ");
      else
        printf ("Il piu' grande e' il terzo! \n\n ");
    }
```


PROBLEMA 3:

Scrivere un programma che prende in **input 3 numeri interi** inseriti da tastiera e li scrive in **output in ordine crescente** utilizzando in totale soltanto **3 confronti** (con operatore $\leq$) tra i numeri stessi all'interno di istruzioni if-else annidate.

No variabili supplementari (solo **3 variabili per i 3 numeri**).

Soluzione: (aspetto la vostra!)

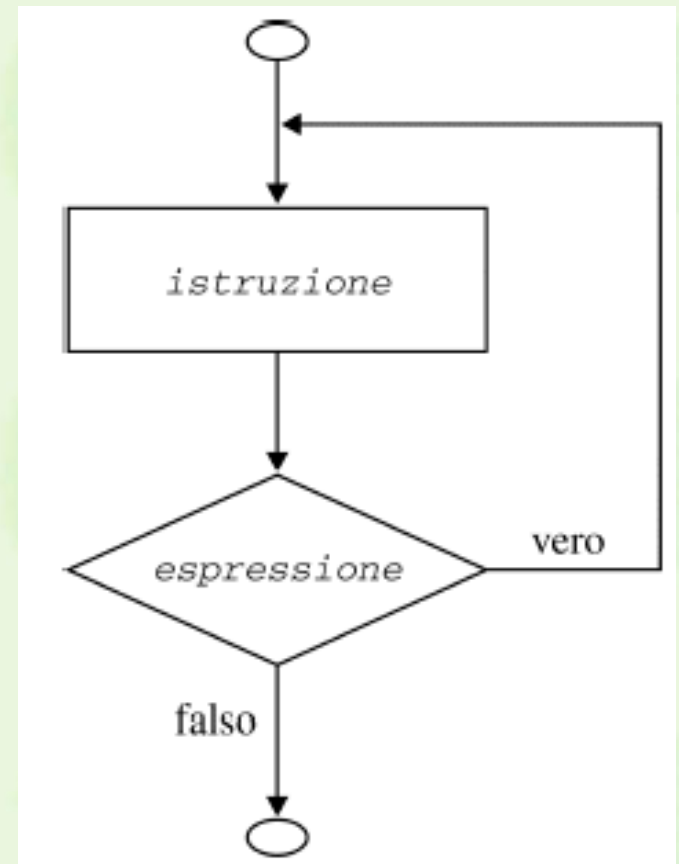
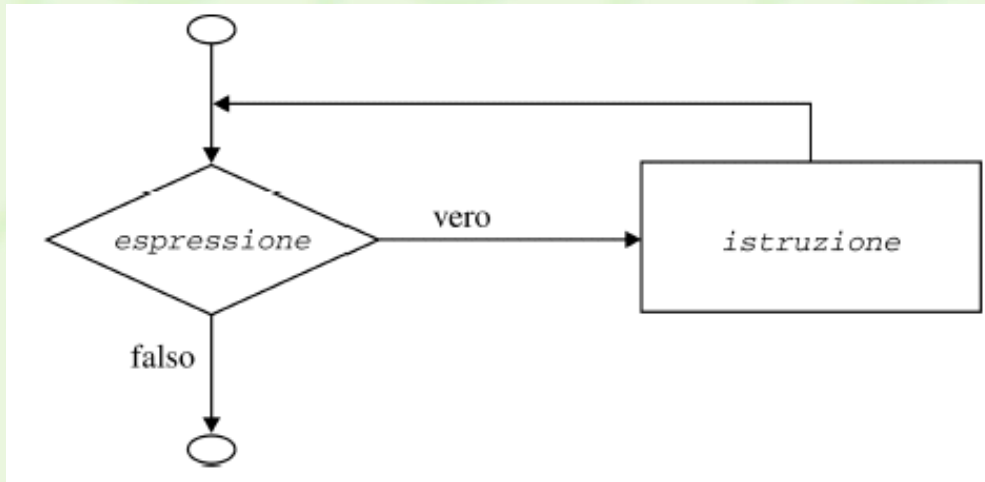
PROBLEMA 4 (**Importante/Difficile**):

Qual è il **minimo** numero di **operazioni di confronto** necessarie per ordinare in ordine crescente **n numeri** interi?

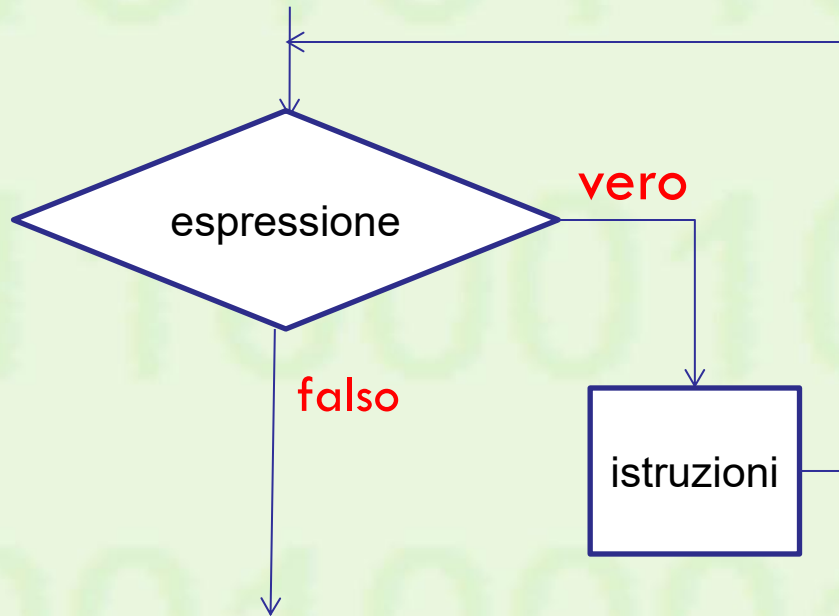
RISPOSTA: è (dell'ordine di) **$n \log n$**

Lower bound per il problema del sorting.

Due possibili schemi per l'iterazione:



Ciclo **while** (a condizione iniziale)



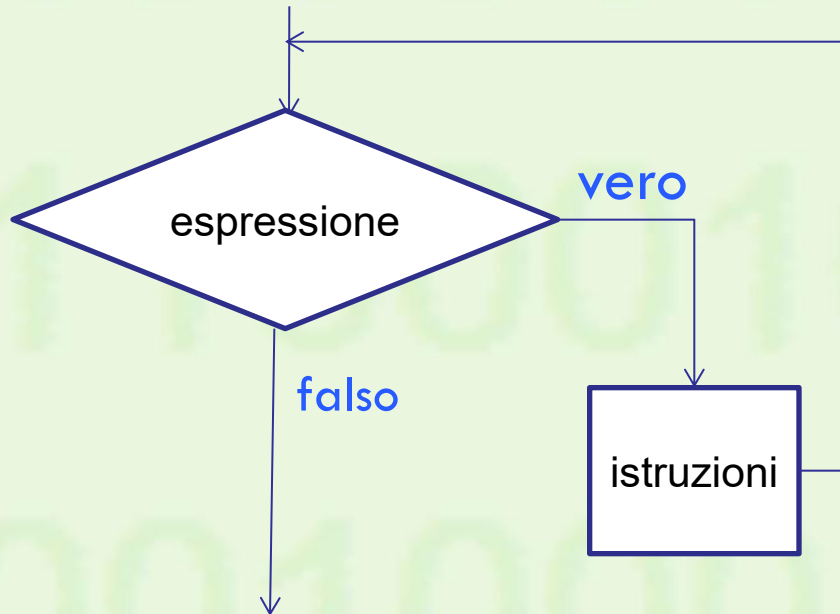
NOTE:

1. è possibile non eseguire mai il blocco
2. le variabili dell'espressione devono essere state inizializzate
3. il blocco deve poter modificare il valore dell'espressione altrimenti verrebbe eseguito indefinitamente (ciclo infinito)
4. Il ciclo termina quando la condizione è falsa

Come funziona:

1. si valuta l'espressione
 2. se ha valore vero: si esegue il blocco di istruzioni e si torna al punto 1
- se ha valore falso: si passa oltre il ciclo

Ciclo **while**



```
while (espressione)  
  istruzione;
```

```
while (espressione)  
{  
  istruzione1;  
  istruzione2;  
}
```

- Se la condizione ha valore **vero**, esegue il corpo e rivaluta la condizione; altrimenti procede oltre il ciclo.
- Ogni istruzione del blocco termina con “;”

Esempi

- ```
x=1;
while (x<5) {
 printf("ciao!\n");
 x=x+1;
}
```
- ```
x=1; // CICLO INFINITO
while (x<5){
    printf("ciao!\n");
}
```
- ```
while (1) { // CICLO INFINITO
 printf("ciao!\n");
}
```
- ```
x=10; //CICLO NULLO
while (x<5){
    printf("ciao!\n");
}
```

Ciclo **do...while** (a condizione finale)



NOTE:

1. il blocco si esegue almeno una volta
2. le variabili dell'espressione devono essere state inizializzate
3. Se l'espressione è vera, il blocco deve poter modificare il valore dell'espressione altrimenti verrebbe eseguito indefinitamente (ciclo infinito)
4. Si esce dal ciclo quando la condizione risulta falsa

Come funziona:

1. si esegue il blocco
2. si valuta l'espressione e se ha valore vero si torna al punto 1, se ha valore falso: si passa oltre il ciclo

Ciclo do...while



```
do istruzione while  
(espressione) ;
```

```
do  
  {  
    istruzione1;  
    istruzione2;  
  }  
while (espressione) ;
```

- Esegue il corpo una volta; se la condizione ha valore vero, lo riesegue e rivaluta la condizione; altrimenti procede oltre il ciclo
- ogni istruzione termina con ;
- l'intero costrutto termina con ;

Istruzione `do..while`

- Sintassi

```
do{  
    <istruzione_1>  
    ...  
    <istruzione_r>  
}  
while (<espressione booleana>  
    <istruzione_x>;  
    <istruzione_y>;  
    .....
```

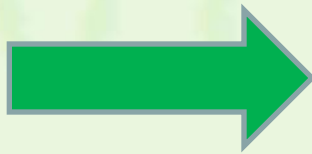
- Significato

Il blocco `<istruzione_1>..<istruzione_r>` viene eseguito **fintanto che** `<espressione booleana>` è *true* (viene eseguito **almeno una volta**).

Equivalenza **while** – **do...while**

- Dato un ciclo **do...while** si può scrivere il ciclo **while** equivalente

```
do  
{  
  istruzioni  
}  
while (espressione) ;
```

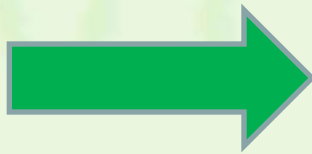


```
{  
  istruzioni  
}  
while (espressione)  
{  
  istruzioni  
}
```

Equivalenza **while** – **do...while**

- I cicli a condizione iniziale e a condizione finale sono equivalenti. Per esempio, dato un ciclo **while** si può scrivere il ciclo **do...while** equivalente

```
while  
  (espressione)  
{  
  istruzioni  
}
```



```
if (espressione)  
{  
  do  
  {  
    istruzioni  
  }  
  while (espressione);  
}
```

```
while (not edge) {  
  run();  
}
```

```
do {  
  run();  
} while (not edge);
```



Operatori incremento/decremento

- Gli operatori incremento **++** e decremento **--** sono operatori unari (serve un unico operando): aggiungono (tolgono) l'unità (1) all'operando.
- Equivalgono ad assegnare alla stessa variabile il proprio valore incrementato
- **x++** **è lo stesso di** **x=x+1**
- **x--** **è lo stesso di** **x=x-1**
- ATTENZIONE: due diverse modalità di esecuzione, qualora l'incremento sia parte di un'istruzione più complessa.
- **x++** ; prima usa x nell'istruzione, poi lo incrementa
- **++x** ; prima incrementa x, poi lo usa nell'istruzione

Operatori incremento decremento

ATTENZIONE!

Le due istruzioni di incremento (decremento) *isolate* hanno lo stesso effetto. Il risultato cambia quando vengono usate in istruzioni meno semplici.

```
int n, m=0;  
n = m++;
```

n vale 0, m vale 1

```
int n, m=0;  
n = ++m;
```

n vale 1, m vale 1

Incrementi e decrementi

L'incremento di un valore diverso da uno
lo si ottiene con +=

```
somma = somma+9;
```

```
somma += 9;
```

La forma generalizzata è

variabile [operatore]= espressione

dove *operatore* può essere + - * / %

La si utilizza quando la variabile appare a sinistra e a destra di un assegnamento.

Forma compatta

```
a *= 5;
```

```
a -= b;
```

```
a *= 4+b;
```

Forma classica

```
a = a*5;
```

```
a = a-b;
```

```
a = a*(4+b);
```

Regole di precedenza

Più alta

- Operatori *unari* $+$, $-$, $!$, $++$, $--$
- Operatori aritmetici *binari* $*$, $/$, $\%$
- Operatori aritmetici binari $+$, $-$
- Operatori relazionali $<$, $>$, $<=$, $>=$
- Operatori di uguaglianza $==$, $!=$
- Operatore booleano $\&\&$
- Operatore booleano $||$
- Operatori assegnamento $=$, $+=$, $-=$, $*=$, $/=$

Più bassa

Il costrutto **for**

E' un costrutto di iterazione come il while e il do while.

Equivalente agli altri due!

Si usa nei casi di **iterazione definita da contatore**

Istruzione **for**

- Sintassi

```
for (<espr1> ; <espr2> ; <espr3> )  
{ <istruzione_1>  
    ...  
    <istruzione_r>  
}  
<istruzione_x>;  
.....
```

Come funziona:

- valuta/ esegue *<espr1>* [inizializzazione]
- valuta (o esegue) *<espr2>*: *se ha valore falso* procede oltre il ciclo; *se ha valore vero*, esegue il corpo e poi valuta (o esegue) *<espr3>* [modifica valori di <espr2>]

Esempi

```
c=1;  
while (c<=n){  
    printf(" * \n");  
    c++;  
}
```

...equivalente a:

```
for (c=1; c<=n; c++)  
    printf("*\n");
```

Quanti * vengono stampati?

Esempi

```
c=0;  
while (c<=n) {  
    c++;  
    printf( \" \"%d\" \", c) ;  
}
```

...equivalente a..

```
for (c=0; c<=n; c++)  
    printf( \" \"%d\\n\" \", c+1) ;
```

Quali numeri vengono stampati?

Esempi di cicli di conteggi

Conteggio da 0 a n:

```
for (i=0;i<n;i++)  
{  
    istruzioni;  
}
```

Conteggio da n a 0:

```
for (i=n;i>0;i--)  
{  
    istruzioni  
}
```

Conteggio da 1 a n:

```
for (i=1;i<=n;i+=1)  
{  
    istruzioni  
}
```

Conteggio da n a 1:

```
for (i=n;i>=1;i=i-1)  
{  
    istruzioni  
}
```

Equivalenza for - while

- I cicli for equivalgono ai cicli while

```
for (espr1 ; espr2 ; espr3)  
{  
    istruzioni  
}
```



```
espr1;  
while (espr2)  
{  
    istruzioni  
    espr3;  
}
```

```
#include <stdio.h>
int main(void)
{
    int count;

    for (count = 1; count <= 500; count++)
        printf("I will not throw paper airplanes in class.");

    return 0;
}
```

AMEND 10-3

NICE TRY.

© 2005 Bill Amend / Contributed by Universal Press Syndicate

Abbiamo visto come l'istruzione **for** risulti utile in molte situazioni in cui c'è da ripetere una istruzione (blocco di istruzioni) un numero di volte definito *a priori*.

Vediamo adesso dei casi particolari di utilizzo.

Istruzione **for**

- Sintassi

```
for (<espr1> ; <espr2> ; <espr3> )  
{ <istruzione_1>  
    ...  
    <istruzione_r>  
}  
<istruzione_x>;  
.....
```

Come funziona:

- valuta/ esegue *<espr1>* [inizializzazione]
- valuta (o esegue) *<espr2>*: *se ha valore falso* procede oltre il ciclo; *se ha valore vero*, esegue il corpo e poi valuta (o esegue) *<espr3>* [modifica valori di *<espr2>*]

Se mancano espressioni nel ciclo **for**

- Se manca *espr1* non viene effettuata l'inizializzazione (deve essere effettuata prima del for)
- Se manca *espr2* il ciclo è infinito (attenzione!)
- Se manca *espr3* l'aggiornamento deve essere gestito nel corpo del ciclo

Nota: mancano = sono istruzioni vuote

Esempio: manca *espr1*

```
c=1;  
...  
for (; c<=n; c++) {  
    printf(" * \n");  
}
```

```
for(c=1; c<=n; c++) {  
    printf(" * \n");  
}
```

Equivalente a:



Esempio: manca *espr2*

```
for (c=1;    ;c++) {  
    printf(" * \n");  
}
```

CICLO INFINITO (LOOP)

Esempi: manca *espr3*

```
for(c=1; c<=n;) {  
    printf(" * \n");  
    c++;  
}
```

Equivalente a:

```
for(c=1; c<=n; c++) {  
    printf(" * \n");  
}
```



Operatore virgola

- Combina più espressioni di qualsiasi tipo
 - *esprA, esprB*
- È associativo a sinistra:
 - si valutano le espressioni da sinistra a destra
 - il valore restituito dall'operatore è il valore dell'ultima espressione
 - ha la priorità più bassa (per cui non occorrono parentesi)
- Di solito si usa per inizializzazioni e aggiornamenti multipli nei cicli **for**

Esempio con doppia variabile di controllo

```
int main(void) {  
    int x ,y;  
    for ( x=0, y=0; x+y<100; x++ , y+=3)  
        printf ("%d ", x+y);  
}
```

Esempi

```
for (a=2, b=1; a<=10 && b<=9; ++a, b=a+b)
    printf("a=%d    b=%d\n",a,b) ;
```

```
for (a=2, b=1, c=0; a<=10 && c<=20; ++a, c=a+b)
    printf("a=%d    b=%d    c=%d\n",a,b,c) ;
```


Esempio (senza le istruzioni !)

```
for(c=0; c<=n; c++)  
    printf("%d\n", c+1);
```

...equivalente a:

```
for(c=0; c<=n; printf("%d\n", c+1), c++ );
```

Ciclo for "senza istruzioni"

- Può avvenire che non occorranzo istruzioni nel corpo di un ciclo perché tutto ha luogo nell'inizializzazione e nell'aggiornamento
 - **for** (*espres1*; *espres2*; *espres3*) ;

Esempio:

```
for(numero=1, somma=0; numero!=0; printf("Inserisci un  
intero:\t), scanf("%d",&numero), somma=somma+numero);
```

ESEMPIO

```
flag=0;
for(d=2; d<n; d++)
    if (n%d==0) {
        flag=1;
        d=n;
    }
if (flag==0) printf(`'primo'');
```

1

2

```
flag=0;
for(d=2; d<n && flag==0; d++)
    if (n%d==0)
        flag=1;
if (flag==0) printf(`'primo'');
```

```
for (d = 2 ; (n % d != 0) && (d < n ) ; d++) ;
if (d == n) printf("primo");
```

3

Interruzioni (!)

break : interrompe l'esecuzione del **case**, provocando un salto del flusso di esecuzione alla prima istruzione successiva, può forzare la terminazione di un'iterazione **for**, **while** o **do-while**

continue: come **break** impedisce l'esecuzione delle istruzioni successive del ciclo ma provoca una nuova iterazione a partire dall'inizio.

exit: della libreria standard **stdlib.h** provoca l'immediata terminazione del programma e il ritorno al sistema operativo.

goto *<etichetta>*: provoca un salto all'istruzione preceduta da *etichetta* (un identificatore C valido) e dal carattere due punti.

Le ragioni della programmazione strutturata, tra cui pulizia ed eleganza del codice, sconsigliano l'uso generalizzato di **break**, **continue** ed **exit**, di cui vedremo in seguito alcuni esempi di utilizzo e “proibiscono” quello di **goto**.

Istruzione **break**

- In C si può uscire da un ciclo in punti diversi dalla condizione di permanenza (ciò è contrario in linea di principio alla programmazione strutturata)
- L'istruzione **break** provoca l'uscita dal costrutto corrente (**switch**, **while**, **do...while** e **for**)
- È sempre sostituibile modificando la condizione

```
for (d = 2 ; d < n ; d++)  
    if (n % d == 0) break;
```

...equivalente a:

```
for (d = 2 ; (d < n) && (n % d != 0) ; d++)
```

Istruzione **continue**

- L'istruzione **continue** provoca il passaggio all'iterazione successiva, ignorando il resto del corpo (si usa solo nei cicli)
- È sempre sostituibile dalle istruzioni **if** o **if...else**

Esempio: Somma i quadrati dei numeri dispari compresi fra 1 e n

```
s = 0;
for (i = 1 ; i <= n ; i++){
    if (i % 2 == 0)
        continue;
    s = s+i*i;
}
```

...equivalente a:

```
s = 0;
for (i = 1 ; i <= n ; i++){
    if (i % 2 != 0)
        s = s+i*i;
}
```

...equivalente a:

```
s = 0;
for (i = 1 ; i <= n ; i=i+2){
    s = s+i*i;
}
```

Cicli annidati

Possiamo (ovviamente!) definire un ciclo dentro un altro ciclo....

```
for(i=1; i<=n; i++)  
    for(j=1; j<=m; j++)  
        istruzione;
```

Ciclo esterno

Ciclo interno

<istruzione> verrà ripetuta $n \cdot m$ volte

Cicli annidati

Per ripetere una determinata *istruzione* $n \cdot m$ volte possiamo scrivere:

```
for(i=1; i<=n; i++)  
    for(j=1; j<=m; j++)  
        istruzione;
```

```
#include <stdio.h>
```

```
main()          /* esempio cicli annidati */  
{  
    int n, m, i, j;  
    printf("Inserire numero linee: ");  
    scanf("%d", &n);  
    printf("Inserire numero colonne: ");  
    scanf("%d", &m);
```

```
    for(i=1; i<=n; i++) {  
        printf("\n");  
        for(j=1; j<=m; j++)  
            printf("(%d,%d) ", i, j );  
        }  
    }
```

Ciclo esterno

Ciclo interno

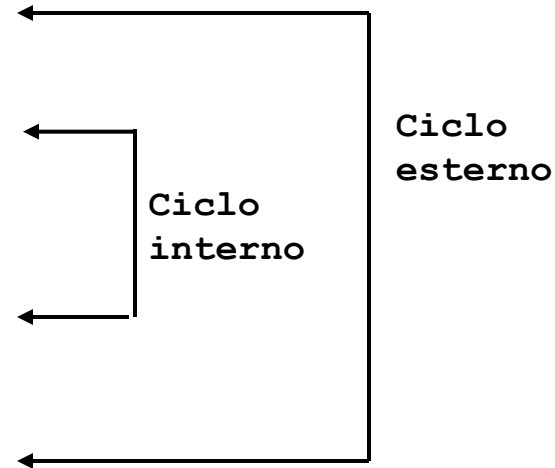
Cicli annidati

```
#include <stdio.h>
int main()
{
    int r;
    int c;

    for( r=10; r >= 1;  r = r - 1)
    {
        for (c=1; c <= 10; c = c + 1)
            if (r % 2 !=0)
                printf( "< " );
            else printf( "> " );

        printf("\n" );
    }

    return(0) ;
}
```

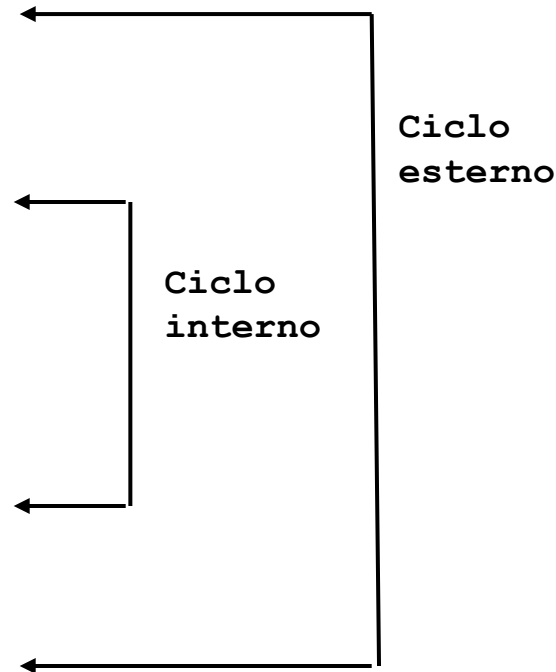


Cicli annidati

(esempio precedente in versione while)

```
#include <stdio.h>
int main()
{
    int r = 10;
    int c;
    while ( r >= 1 )
    {
        c=1;
        while ( c <= 10 ) {
            if (r % 2 !=0)
                printf( "< " );
            else printf( "> " );
            c = c + 1;
        }
        r = r - 1;
        printf("\n" );
    }

    return(0);
}
```



Cicli annidati

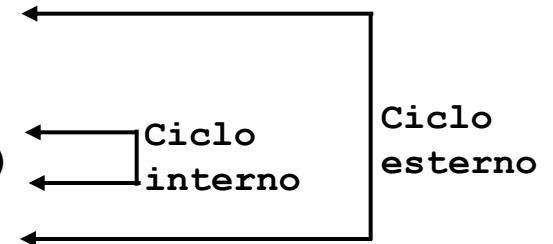
Per ripetere una determinata *istruzione* $n \cdot m$ volte possiamo scrivere:

```
for(i=1; i<=n; i++)  
    for(j=1; j<=m; j++)  
        istruzione;
```

```
#include <stdio.h>
```

```
main()          /* esempio cicli annidati */  
{  
    int n, m, i, j;  
    printf("Inserire numero righe: \t");  
    scanf("%d", &n);  
    printf("Inserire numero colonne: \t");  
    scanf("%d", &m);
```

```
    for(i=1; i<=n; i++) {  
        printf("\n");  
        for(j=1; j<=m; j++)  
            printf("*");  
    }  
}
```



Cicli annidati

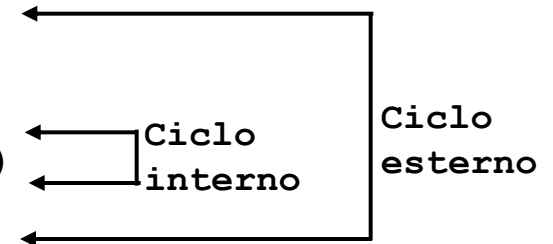
Per ripetere una determinata *istruzione* $n \cdot m$ volte possiamo scrivere:

```
for(i=1; i<=n; i++)  
    for(j=1; j<=m; j++)  
        istruzione;
```

```
#include <stdio.h>
```

```
main()          /* esempio cicli annidati */  
{  
    int n, m, i, j;  
    printf("Inserire numero righe: \t");  
    scanf("%d", &n);  
    printf("Inserire numero colonne: \t");  
    scanf("%d", &m);
```

```
    for(i=1; i<=n; i++) {  
        printf("\n");  
        for(j=1; j<=m; j++)  
            printf("*");  
    }  
}
```



Istruzione break nei cicli annidati

```
#include <stdio.h>

main()      /* esempio cicli annidati */
{
    int n, m, i, j;
    printf("Inserire numero linee: \t");
    scanf("%d", &n);
    printf("Inserire numero colonne: \t");
    scanf("%d", &m);

    for(i=1; i<=n; i++) {
        printf("\n");
        for(j=1; j<=m; j++){
            if (j==3)
                break;
            printf("+");
        }
    }
}
```

